



Titel: Eine saubere Architektur, bei der die Quelle der Wahrheit in der Datenbank liegt.

Author: Jeff Meridian

Eine saubere Architektur, bei der die Quelle der Wahrheit in der Datenbank liegt. Dieses Dokument bietet einen detaillierten Überblick über die Testsuite, die Codierungsagenten dabei helfen soll, die Architektur, Mocking-Strategien und Funktionsabdeckung der lokalen Umgebungstests zu verstehen.
`jeffmeridianlocal_sanity.spec.ts`

Übersicht

Die ist eine umfassende Playwright-basierte Testsuite, die die lokale Entwicklungsumgebung für das Writer Studio und die zugehörige von Eleventy generierte Website validiert. Es dient sowohl als UI-Sanity-Check als auch als Backend-Tool-Integrationsüberprüfung.
`jeffmeridianlocal_sanity.spec.ts`

Primäres Ziel: `http://localhost:8080`

Wartungshinweise für Agenten

- **Hinzufügen neuer Plugins:** Wenn Sie ein neues Plugin hinzufügen `anti/plugin/` Stellen Sie sicher, dass es die `umgebungsvARIABLE`, um die Kompatibilität mit dieser Suite aufrechtzuerhalten.
`TESTDBFILE`
- **Schemaänderungen:** Wenn sich das Datenbankschema weiterentwickelt, aktualisieren Sie das `sql` Zeichenfolge in der obersten Ebene `test.beforeAll` Block zum Einbinden neuer Tabellen oder Spalten.
- **Bereinigung:** Die Suite entbindet automatisch die Dateien nach der Ausführung. Speichern Sie keine persistenten Daten in diesen Dateien.
`testmock*.db`

Voraussetzungen

Um diese Tests erfolgreich durchführen zu können, muss Folgendes auf dem System verfügbar sein:

1. **Node.js & Dramatiker:** `npx playwright test`
1. **Python 3:** Zur Ausführung der Plugins und `plugin_host.py`.
1. **SQLite3 CLI:** Muss installiert und verfügbar sein in der `$PATH` für das Mock-Datenbank-Seeding.
1. **Lokaler Server:** Der Eleventy-Dev-Server sollte laufen unter `http://localhost:8080`.

Durchführung der Tests

So führen Sie die vollständige Suite aus:

```
npx playwright test transcripts/tests/
jeff_meridian_local_sanity.spec.ts
```

Um nur die Integrationstests des Python-Tools auszuführen:

```
npx playwright test -g "Anti Plugin
Tools|Writer Studio Tools API"
```

1. Mocking-Strategie (SQLite-Isolierung)

Um sicherzustellen, dass Tests zerstörungsfrei und idempotent sind, verwendet die Suite **Isolierung auf Datenbankebene**.

- **Arbeitnehmerspezifische Datenbanken:** Zu Beginn des Testlaufs (`test.beforeAll`) generiert jeder Playwright-Mitarbeiter eine eindeutige SQLite-Datei (z. B. `testmock0.db`)
- **Aussaat:** Die Mock-Datenbank wird mit dem `sqlite3` CLI mit einem standardisierten Datensatz:
- A `documents` Tabelle mit einem „Mock Doc“ (ID 999).
- Ein `app_state` Tabelle mit dem aktiven Dokumentstatus auf „Mock Doc“ gesetzt.
- **Umweltinjektion:** Der Pfad zu dieser Mock-Datenbank wird an alle untergeordneten Prozesse (Python-Skripte und den MCP-Host) übergeben über die Umgebungsvariable `TESTDBFILE`

2. Testgruppierung

Die Suite ist in mehrere Logikblöcke unterteilt:

A. Homepage Sanity & Abschnittssichtbarkeit

Bestätigt, dass der Eleventy-Build korrekt ist und alle wichtigen UI-Abschnitte vorhanden sind.

- **Generische Tests:** Anstatt nach bestimmten Artikeliteln zu suchen, überprüft es die Existenz von Strukturklassen wie `.hero-featured`, `.app-icon-card`, und `.post-card`.
- **Widerstandsfähigkeit:** Tests überprüfen, ob dies der Fall ist *Mindestens einer* Element in jedem Raster, um sicherzustellen, dass der Test nicht fehlschlägt, nur weil ein bestimmter Artikel umbenannt oder entfernt wurde.

B. Sprachwechsel und Persistenz

Testet die Internationalisierungslogik (i18n).

- Überprüft, ob das Klicken auf Flags die URL aktualisiert (z. B. `/?lang=de`).
- Bestätigt, dass die Sprachpräferenz beibehalten wird in `localStorage` und beim Neuladen einer Seite korrekt angewendet.

C. Robuste Navigation (Spiele & Blog)

Testet die Linkintegrität auf der gesamten Site.

- **Generischer Link folgt:** Die Tests finden dynamisch den ersten verfügbaren Spiel- oder Blog-Link, klicken darauf und

überprüfen, ob die resultierende URL korrekt ist und der Seitentext erfolgreich geladen wird.

- **Header-Prüfung:** Es erfordert nicht unbedingt eine auf Spielseiten und bietet Platz für benutzerdefinierte Spiele-Engines, die möglicherweise eine Vollbild-Leinwand verwenden.

D. Writer Studio Tools API (MCP-Integration)

Validiert den Python-basierten MCP-Server (Model Context Protocol).

- **Prozess-Spawning:** Es spawnt `plugin_host.py` als Kindprozess.
- **JSON-RPC-Kommunikation:** Kommuniziert mit dem Host über `stdin/stdout` Verwendung standardmäßiger JSON-RPC-Nutzlasten.
- **Werkzeugabdeckung:** Übt die , , und Tools gegen die Mock-Datenbank `searchdocumentsbatchoperationsreplace_text`

E. Anti-Plugin-Tools (Direkte Ausführung)

Testet direkt die eigenständigen Python-Plugins, die für die aktive Dokumentenverwaltung verwendet werden.

- Überprüft, dass , , und Richtig mit dem SQLite interagieren
- **Serialisierung:** Dieser Block verwendet `Tisch.readactivatedocument.py` und `test.description.serial` um gleichzeitige Schreibkollisionen in der Mock-Datenbank des Arbeiters zu verhindern.